Aakaar Backend — Container & Deployment

FastAPI service (**aakar.api.main:app**) packaged on the official Microsoft Playwright Python base image. Single-process, single-worker by design: an in-process FAISS index and a per-run browser pool make horizontal scaling the only safe path beyond one replica.

1. Image at a glance

Field	Value
Base image	mcr.microsoft.com/playwright/python:v1.47.0-jammy
Python	3.11+ (provided by base image)
Working dir	/app
User	aakar (non-root, uid auto-assigned)
Exposed port	8000/tcp
Persistent volume	/data (FAISS index, capability cache, run artifacts)
Entrypoint	alembic upgrade head && uvicorn aakar.api.main:app
Healthcheck	GET http://127.0.0.1:8000/health (every 30s)
Workers	1 (FAISS + Playwright pool not fork-safe)

2. Dockerfile

Path: **aakar/Dockerfile**. The runtime image installs Python deps in a dedicated layer so subsequent code edits do not retrigger a 5–7 minute *sentence-transformers* + *faiss-cpu* rebuild.

```
# syntax=docker/dockerfile:1.7
# ---- Aakaar backend image ------------------------------------------------
# Uses Microsoft's Playwright base which already ships Chromium and the
# system libraries Playwright requires — saving ~500 MB of apt installs.
FROM mcr.microsoft.com/playwright/python:v1.47.0-jammy AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    AAKAR_DATA_DIR=/data \
    OMP_NUM_THREADS=1 \
    OPENBLAS_NUM_THREADS=1 \
    MKL_NUM_THREADS=1 \
    TOKENIZERS_PARALLELISM=false \
    LOKY_MAX_CPU_COUNT=1

WORKDIR /app

# Install Python deps first so the dep layer caches across code edits.
COPY pyproject.toml ./
RUN python -m pip install --upgrade pip \
 && pip install --no-cache-dir \
        "pydantic>=2.6,<3" "sqlalchemy>=2.0,<3" "alembic>=1.13" \
        "numpy>=1.26" "faiss-cpu>=1.8" "openai>=1.40" \
        "sentence-transformers>=2.7" "fastapi>=0.115" "uvicorn[standard]>=0.30" \
        "bcrypt>=4.1" "PyJWT>=2.9" "python-multipart>=0.0.9" \
        "httpx>=0.27" "python-dotenv>=1.0" "playwright>=1.47" \
 && playwright install --with-deps chromium

# Copy source.
COPY aakar ./aakar
```

```
COPY alembic.ini ./alembic.ini
COPY interpreter ./interpreter
COPY tests ./tests

# Non-root runtime user.
RUN groupadd --system aakar \
 && useradd  --system --gid aakar --home /app aakar \
 && mkdir -p /data \
 && chown -R aakar:aakar /app /data
USER aakar

EXPOSE 8000
VOLUME ["/data"]

HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
  CMD curl -fsS http://127.0.0.1:8000/health || exit 1

# Run migrations then boot uvicorn. Single worker is intentional — the
# in-process FAISS index and per-run browser pool are not fork-safe.
CMD alembic upgrade head && \
    exec uvicorn aakar.api.main:app \
      --host 0.0.0.0 --port 8000 --workers 1
```

3. Required environment variables

Name	Purpose
AAKAR_JWT_SECRET	Long random string (>=32 bytes). Refuses to start without it.
AAKAR_DB_URL	SQLAlchemy URL. Defaults to sqlite under /data; use postgresql+psycopg in prod.
AAKAR_DATA_DIR	Mount target for persistent state (default /data).
AAKAR_CORS_ORIGINS	Comma-separated allowlist of frontend origins.
AAKAR_SUPERUSER_EMAIL	Bootstraps the platform superuser on first migration.
AAKAR_SUPERUSER_PASSWORD	Pair with the above. Read once at startup.
OPENAI_API_KEY	Required for the agentic planner (NL→DAG).
AAKAR_LLM_MODEL	Defaults to gpt-4.1-mini.
AAKAR_BROWSER_HEADLESS	true in container; set false only for local debugging.
AAKAR_LIVE_SCREENSHOTS	Toggle live run screenshots (storage cost vs. UX).

4. Build & run

Build the image from the repo root:

```
docker build -t aakaar/api:latest -f aakar/Dockerfile aakar
```

Run a single replica against an external Postgres:

```
docker run --rm -p 8000:8000 \
  -e AAKAR_JWT_SECRET=$(openssl rand -hex 32) \
  -e AAKAR_DB_URL=postgresql+psycopg://aakar:aakar@db.internal:5432/aakar \
  -e OPENAI_API_KEY=$OPENAI_API_KEY \
  -e AAKAR_CORS_ORIGINS=https://app.aakaar.example \
  -v aakar-data:/data \
  aakaar/api:latest
```

5. Database migrations

The container runs **alembic upgrade head** on every boot before uvicorn starts. This is idempotent and safe to run on each replica when only one replica is alive.

For multi-replica rollouts, run migrations as a separate Job (Kubernetes) or one-shot task (ECS) *before* rolling the deployment, then disable the inline upgrade with **command: ["uvicorn", ...]**. This avoids two replicas racing the migration lock.

6. Resource sizing

Profile	Recommended limits
Dev / smoke	0.5 vCPU, 1.5 GiB memory
Staging	1 vCPU, 2.5 GiB memory
Production (single tenant)	2 vCPU, 4 GiB memory
Production (10+ tenants)	4 vCPU, 8 GiB memory + 2nd replica

Memory is dominated by **sentence-transformers** (BGE-small) loaded once at import (≈700 MiB resident) and Chromium browser instances (≈180 MiB each). Cap the per-tenant browser pool via **AAKAR_BROWSER_POOL_MAX** if you co-locate replicas.

7. Persistence & state

• **/data** volume is required — loss wipes capability embeddings, run artifacts, and (if AAKAR_DB_URL is sqlite) the entire database.

• Database of record is Postgres-compatible (Yugabyte in production). pgvector is used for capability search; FAISS is the dev fallback when sqlite is the DB.

• Browser sessions are pinned per run, never reused across runs. Storage state is written to /data/sessions/<tenant>/<run_id>/.

8. Observability

• Liveness: GET /health returns 200 once the app finishes loading models.

• Logs: structured JSON to stdout via aakar.shared.logging_setup. Ship via the platform log driver (CloudWatch, Loki, etc.).

• Metrics: uvicorn access logs + per-run timeline events stored in Postgres. Surface p95/p99 latency from the load balancer.

9. Security notes

• Container runs as the non-root **aakar** user; /data is the only writable mount.

• Playwright executes user-driven web logins. Run the API in a network namespace with egress only to required tenant URLs — do not expose internal services.

• Credentials live in the capability registry/vault, never in chat. The planner is instructed to *request* rather than collect secrets.

• Rotate AAKAR_JWT_SECRET via a rolling restart; outstanding tokens become invalid.